

PDF RAG System — Approach & Methodology

Test document: Apple Inc. Form 10-Q, Q3 2022

What was built

A fully local Retrieval-Augmented Generation system that answers questions over a PDF's text, tables, and figures. No external API keys are used — all embedding, vision, and generation models run locally via Ollama.

Pipeline

- **Ingest** — PyMuPDF extracts page text, detects tables and converts each to markdown, and pulls embedded images above a size threshold (filters logos/icons).
- **Caption figures** — each extracted image is captioned by a vision-language model (qwen3-vl:8b), turning it into searchable text describing labels/values shown.
- **Chunk** — page text is split into overlapping ~800-character chunks; tables are kept as whole markdown blocks (split only if very long); every chunk is tagged with page number, detected section heading, and type (text/table/figure).
- **Embed & index** — all chunks are embedded with bge-m3 and stored in a local Chroma vector store, persisted to disk.
- **Retrieve & answer** — a query is embedded and matched against all chunk types (no separate table/figure index), top-k chunks are passed as context to qwen2.5:7b, which is instructed to answer only from that context and cite page numbers.
- **Interfaces** — CLI (src/rag.py) and a Streamlit UI (src/app.py) showing the answer plus its source chunks (type + page + section).

Design decisions

Decision	Reasoning
Local models (Ollama) over a hosted API	No API key dependency; reproducible and free to run; this machine already had bge-m3, qwen2.5:7b and qwen3-vl:8b available.
Single mixed vector index (not separate stores per type)	Most questions don't specify whether the answer is in prose or a table; a unified similarity search with a type tag in metadata is simpler and works well in testing.
Tables converted to markdown, not flattened text	Preserves row/column structure so the LLM can read cell alignment correctly instead of losing numbers in a wall of text.
Figures filtered by pixel size before captioning	Avoids wasting a VLM call captioning logos/decorative marks; only substantive images are indexed.
PyMuPDF for both text and table extraction	Built-in table detection (find_tables) is accurate enough for this filing and avoids extra dependencies (e.g. Camelot, which needs Ghostscript on Windows).

Assumptions

- The test PDF (a 10-Q filing) is representative: text- and table-heavy, with no meaningful charts/graphs (it contains a single small embedded logo, filtered out as noise).

- The figure-captioning path was validated on the pipeline logic and the vision model directly; it did not get exercised against a real chart in this document because none exists.
- Single-document scope — the index is rebuilt per PDF rather than designed for multi-document ingestion.

Challenges

- Distinguishing real section headings from all-caps boilerplate (e.g. recurring risk-factor labels) during text chunking — solved with a lightweight heuristic (all-caps short lines), which is adequate but not exact.
- Tables split across page boundaries or with merged cells produced some duplicate columns in raw extraction; handled by keeping the header row with every chunk so context is never lost, even if formatting is imperfect.

Possible improvements

- Render full pages to images and run the VLM on pages with vector-drawn charts (not just embedded raster images), for documents that use native chart drawing.
- Heading detection via font-size/style metadata instead of an all-caps text heuristic, for more reliable section citations.
- Query-time routing (e.g. detect "table" or "figure" in the question) to bias retrieval toward that chunk type when helpful.
- Swap in a larger/instruction-tuned local model or a hosted API as an optional backend for higher answer quality.